

Phase 2 Integration Guide

Overview

This guide explains how to integrate Phase 2 with Phase 1 and prepare for Phase 3-4.

Integration with Phase 1

Architecture Overview

Phase 2 extends Phase 1's lock-free foundations:

Phase 1 (Foundation)		Phase 2 (Memory & Scheduling)
— Lock-free rings	→	— NUMA-aware ring allocation
— Fiber scheduler	→	— Tickless fiber scheduling
— Arena allocator	→	— Per-CPU NUMA arenas
— Graph calls	→	— Prefetch graph targets
— Zero-copy IPC	→	— NUMA-aware zero-copy

Integration Points

1. NUMA-Aware Ring Buffers

Allocate ring buffers from local NUMA node:

```
#include "spsc_ring.h"
#include "per_cpu_arena.h"

// Allocate ring memory from local NUMA node
size_t ring_size = 4096;
void* ring_mem = per_cpu_arena_alloc(ring_size);

// Create ring from NUMA-local memory
spsc_ring_t* ring = spsc_ring_create_from_memory(ring_mem, ring_size);

// Use ring as normal
spsc_ring_enqueue(ring, data);
```

2. Huge Page Fiber Stacks

Use huge pages for fiber stacks to reduce TLB pressure:

```

#include "fiber.h"
#include "huge_pages.h"

// Allocate 2MB huge page for fiber stack
void* stack = huge_page_alloc(HUGE_PAGE_TYPE_2MB);

// Create fiber with huge page stack
fiber_t* fiber = fiber_create_with_stack(entry_func, arg,
                                         stack, HUGE_PAGE_2MB);

// Fiber runs with reduced TLB misses
fiber_run(fiber);

// Cleanup
fiber_destroy(fiber);
huge_page_free(stack, HUGE_PAGE_TYPE_2MB);

```

3. Tickless Fiber Scheduler

Integrate timer wheel with fiber scheduler:

```

#include "fiber_scheduler.h"
#include "timer_wheel.h"

// Create per-CPU timer wheel
timer_wheel_t* wheel = timer_wheel_create();

// Schedule fiber with timeout
void timeout_callback(void* arg) {
    fiber_t* fiber = (fiber_t*)arg;
    fiber_scheduler_unblock(scheduler, fiber);
}

// Block fiber with timeout
timer_id_t timeout = timer_wheel_add(wheel, 100, timeout_callback, fiber);
fiber_scheduler_block(scheduler, NULL);

// Cancel timeout if fiber unblocks early
timer_wheel_cancel(wheel, timeout);

```

4. Prefetch Graph Calls

Prefetch graph call targets before dispatch:

```

#include "graph_call.h"
#include <x86intrin.h>

// Prefetch next graph call target
void graph_call_dispatch_with_prefetch(graph_call_t* call) {
    // Prefetch target capability
    _mm_prefetch((const char*)call->target_cap, _MM_HINT_T0);

    // Prefetch target descriptor
    _mm_prefetch((const char*)call->target_desc, _MM_HINT_T0);

    // Dispatch call
    graph_call_dispatch(call);
}

```

5. NUMA-Aware Zero-Copy IPC

Allocate zero-copy buffers from local NUMA node:

```
#include "zero_copy.h"
#include "per_cpu_arena.h"

// Allocate zero-copy buffer from local node
void* buffer = per_cpu_arena_alloc(buffer_size);

// Create zero-copy descriptor
zero_copy_desc_t* desc = zero_copy_create(buffer, buffer_size);

// Send via IPC
ipc_send(desc);

// Receiver gets NUMA-local buffer
```

Initialization Order

Correct initialization order is critical:

```
// 1. Initialize Phase 1
phase1_init();

// 2. Initialize Phase 2
phase2_config_t config = {
    .integrate_phase1 = true,
    // ... other config
};
phase2_init(&config);

// 3. Create integrated components
// Now you can use Phase 1 + Phase 2 together
```

Performance Considerations

NUMA Locality:

- Phase 1 rings: Allocate from local node
- Phase 1 arenas: Replace with per-CPU arenas
- Phase 1 fibers: Use huge page stacks

TLB Efficiency:

- Use huge pages for large Phase 1 allocations
- Enable PCID for fiber context switches
- Batch TLB invalidations across Phase 1 operations

Timer Overhead:

- Replace Phase 1 timeouts with timer wheel
- Use tickless operation for idle fibers
- Coalesce timer IPIs across CPUs

Preparing for Phase 3-4

Phase 3 Integration Points

Phase 3 will add hardware acceleration:

```
// Placeholder for Phase 3 DMA integration
typedef struct {
    void* dma_buffer;
    size_t dma_size;
    uint32_t numa_node;
} dma_desc_t;

// Allocate DMA buffer from NUMA node
dma_desc_t* dma_alloc_numa(size_t size, uint32_t node) {
    // Phase 2 provides NUMA-aware allocation
    void* buffer = per_cpu_arena_alloc_cpu(node * 16, size);

    // Phase 3 will add DMA mapping
    // dma_map(buffer, size);

    return create_dma_desc(buffer, size, node);
}
```

Phase 4 Integration Points

Phase 4 will add distributed NUMA:

```
// Placeholder for Phase 4 remote NUMA
typedef struct {
    uint32_t local_node;
    uint32_t remote_node;
    uint32_t remote_machine;
} remote_numa_desc_t;

// Allocate from remote machine's NUMA node
void* remote_numa_alloc(uint32_t machine, uint32_t node, size_t size) {
    // Phase 2 provides local NUMA awareness
    // Phase 4 will extend to remote machines

    // For now, allocate locally
    return per_cpu_arena_alloc(size);
}
```

Migration Guide

Migrating from Phase 1 Only

Step 1: Update includes

```
// Before
#include "shared_arena.h"

// After
#include "shared_arena.h"
#include "per_cpu_arena.h"
```

Step 2: Replace allocations

```
// Before
void* ptr = shared_arena_alloc(arena, size);

// After
void* ptr = per_cpu_arena_alloc(size);
```

Step 3: Update initialization

```
// Before
shared_arena_t* arena = shared_arena_create(size);

// After
per_cpu_arena_init();
```

Migrating Fiber Scheduler

Step 1: Add timer wheel

```
// Before
fiber_scheduler_t* sched = fiber_scheduler_create(cpu_id);

// After
fiber_scheduler_t* sched = fiber_scheduler_create(cpu_id);
timer_wheel_t* wheel = timer_wheel_create();
```

Step 2: Replace timeouts

```
// Before
fiber_scheduler_block_with_timeout(sched, fiber, timeout_ms);

// After
timer_id_t id = timer_wheel_add(wheel, timeout_ms,
                                timeout_callback, fiber);
fiber_scheduler_block(sched, fiber);
```

Migrating Ring Buffers

Step 1: Allocate from NUMA node

```
// Before
spsc_ring_t* ring = spsc_ring_create(capacity);

// After
void* mem = per_cpu_arena_alloc(ring_size);
spsc_ring_t* ring = spsc_ring_create_from_memory(mem, ring_size);
```

Testing Integration

Unit Tests

Test each integration point:

```
// Test NUMA-aware ring allocation
void test_numa_ring(void) {
    per_cpu_arena_init();

    void* mem = per_cpu_arena_alloc(4096);
    assert(mem != NULL);

    spsc_ring_t* ring = spsc_ring_create_from_memory(mem, 4096);
    assert(ring != NULL);

    // Verify NUMA locality
    int node = numa_topology_current_node();
    assert(is_local_memory(mem, node));

    spsc_ring_destroy(ring);
    per_cpu_arena_free(mem, 4096);
}
```

Integration Tests

Test combined Phase 1 + Phase 2:

```
// Test fiber with huge page stack
void test_fiber_huge_stack(void) {
    huge_page_init(NULL);

    void* stack = huge_page_alloc(HUGE_PAGE_TYPE_2MB);
    assert(stack != NULL);

    fiber_t* fiber = fiber_create_with_stack(test_func, NULL,
                                              stack, HUGE_PAGE_2MB);
    assert(fiber != NULL);

    fiber_run(fiber);

    fiber_destroy(fiber);
    huge_page_free(stack, HUGE_PAGE_TYPE_2MB);
}
```

Performance Tests

Measure integrated performance:

```
// Benchmark Phase 1 + Phase 2
void bench_integrated(void) {
    phase1_init();
    phase2_init(NULL);

    // Measure combined performance
    double start = get_time();

    for (int i = 0; i < ITERATIONS; i++) {
        // Use Phase 1 + Phase 2 together
        void* ptr = per_cpu_arena_alloc(64);
        spsc_ring_enqueue(ring, ptr);
        fiber_scheduler_yield(sched);
        timer_wheel_tick(wheel);
    }

    double end = get_time();
    printf("Integrated performance: %.2f ops/sec\n",
        ITERATIONS / (end - start));
}
```

Best Practices

NUMA Awareness

1. Always check NUMA availability:

```
if (numa_topology_available()) {
    // Use NUMA-aware allocation
} else {
    // Fallback to regular allocation
}
```

1. Pin threads to NUMA nodes:

```
int node = numa_topology_current_node();
// Keep thread on same node for locality
```

1. Monitor NUMA statistics:

```
per_cpu_arena_stats_t stats;
per_cpu_arena_get_stats(&stats);

if (stats.remote_allocs > stats.local_allocs * 0.1) {
    // Too many remote allocations, investigate
}
```

Huge Pages

1. Use for large allocations:

```

if (size >= 512 * 1024) {
    // Use huge page
    ptr = huge_page_alloc(HUGE_PAGE_TYPE_2MB);
} else {
    // Use regular allocation
    ptr = per_cpu_arena_alloc(size);
}

```

1. Monitor TLB efficiency:

```

huge_page_stats_t stats;
huge_page_get_stats(&stats);

// Check promotion/demotion balance
if (stats.demotions > stats.promotions) {
    // Memory pressure, consider tuning
}

```

Timer Management

1. Use timer wheel for many timers:

```

if (num_timers > 100) {
    // Use timer wheel
    timer_wheel_add(wheel, delay, callback, arg);
} else {
    // Simple timeout is fine
}

```

1. Coalesce timers:

```

// Instead of exact timeouts, use ranges
timer_wheel_add(wheel, delay ± 10%, callback, arg);

```

Troubleshooting

NUMA Issues

Problem: High remote allocation rate

Solution:

```

// Check CPU affinity
int cpu = numa_topology_current_cpu();
int node = numa_topology_cpu_to_node(cpu);

// Verify thread is on correct CPU
// Use pthread_setaffinity_np() if needed

```

TLB Issues

Problem: High TLB miss rate

Solution:


```
// Enable huge pages
huge_page_config_t config = {
    .enable_2mb = true,
    .promotion_threshold = 256 * 1024
};
huge_page_init(&config);

// Use huge pages for large allocations
```

Timer Issues

Problem: High timer overhead

Solution:

```
// Use coarser granularity
#define TIMER_WHEEL_LEVEL0_TICK_MS 2

// Batch timer operations
// Add multiple timers before ticking
```

Further Reading

- [ARCHITECTURE.md](#) (ARCHITECTURE.md): Detailed architecture
- [API.md](#) (API.md): Complete API reference
- [PERFORMANCE.md](#) (PERFORMANCE.md): Performance analysis
- Phase 1 documentation: `../phase1/docs/`

Support

For integration issues:

1. Check test suite: `tests/test_integration.c`
2. Review examples: `bench/bench_phase2.c`
3. Open GitHub issue with details